[1부] 머신러닝
2장 머신러닝 프로젝트 처음부터 끝까지

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 2장: 머신러닝 프로젝트 처음부터 끝까지

2.1 실제 데이터로 작업하기
2.2 큰 그림 보기
2.3 데이터 가져오기
2.4 데이터 이해를 위한 탐색과 시각화
2.5 머신러닝 알고리즘을 위한 데이터 준비
2.6 모델 선택과 훈련
2.7 모델 세부 튜닝
2.8 론칭, 모니터링, 시스템 유지 보수
2.9 직접 해보세요!

2장 머신러닝 프로젝트 처음부터 끝까지

주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습

큰 그림 → 데이터 구하기 → 데이터 탐색 시각화 → 데이터 준비 → 선택 훈련 → 모델 조정 → 솔루션 제시 → 시스템 론칭 모니터링 유지 보수

2.1 실제 데이터로 작업하기(1)

- 머신러닝을 배울 때는 인공 데이터셋이 아닌 실제 데이터로 실험해보는 것이 가장 좋음
  - 공개 데이터셋 구하기
    - 유명한 공개 데이터 저장소
      - OpenML(https://openml.org)
      - 캐글(https://kaggle.com/datasets)
      - PapersWithCode(https://paperswithcode.com/datasets)
      - UC 어바인 머신러닝 저장소(https://archive.ics.uci.edu/ml)
      - 아마존 AWS 데이터셋(https://registry.opendata.aws)
      - 텐서플로 데이터셋(https://tensorflow.org/datasets)
    - 메타 포털(공개 데이터 저장소가 나열되어 있는 페이지)
      - 데이터 포털(https://dataportals.org)
      - 오픈 데이터 모니터(https://opendatamonitor.eu)
    - 인기 있는 공개 데이터 저장소가 나열되어 있는 다른 페이지
      - 위키백과 머신러닝 데이터셋 목록(https://homl.info/9)
      - Quora(https://homl.info/10)
      - 데이터셋 서브레딧(https://www.reddit.com/r/datasets)

학습에는 StatLib 저장소에 있는 캘리포니아 주택 가격(California Housing Prices) 데이터셋을 사용

2.1 실제 데이터로 작업하기(2)

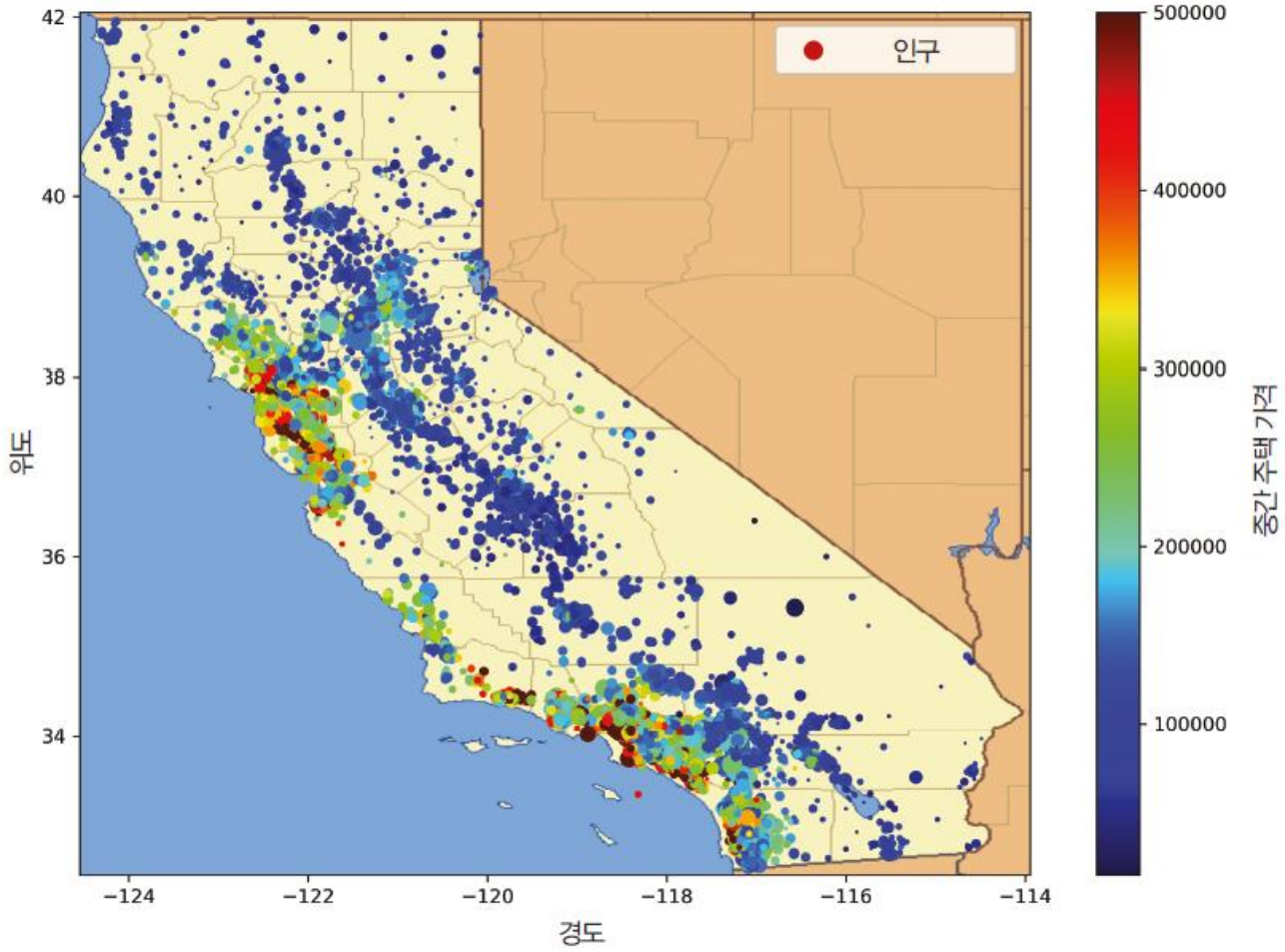

그림 2-1 캘리포니아 주택 가격

2.2 큰 그림 보기(1)

2.2.1 문제 정의

- 비즈니스의 목적이 정확히 무엇인가?'
- 회사에서는 이 모델을 어떻게 사용해 이익을 얻으려고 할까?
- 현재 솔루션은 어떻게 구성되어 있나?

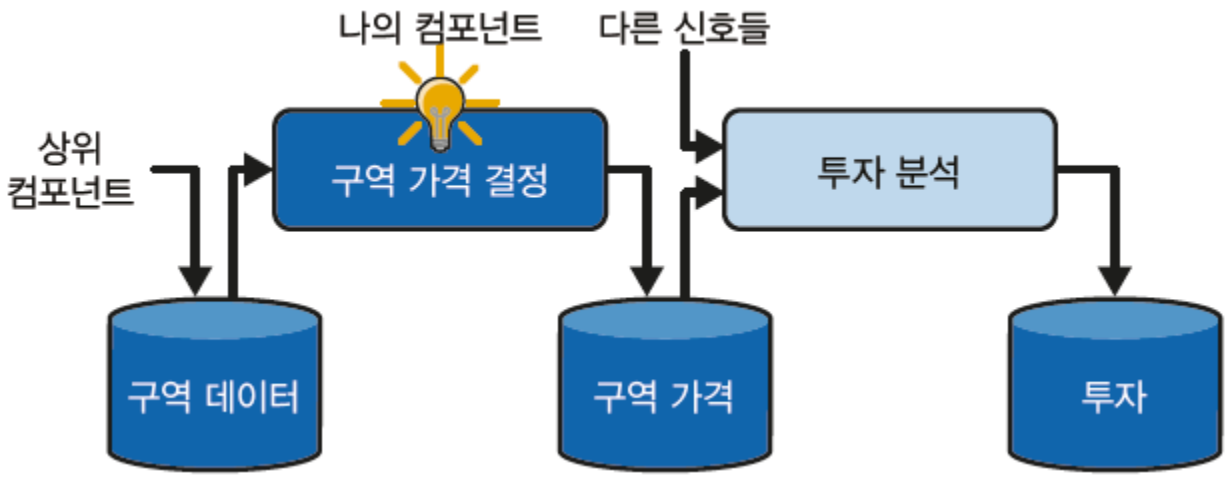

그림 2-2 그림 2-2 부동산 투자를 위한 머신러닝 파이프라인

2.2 큰 그림 보기(2)

- 회사는 구역의 데이터를 기반으로 중간 주택 가격을 예측하는 모델을 훈련시키는 쪽이 유용
- 정보를 취합해 시스템 설계
  - 레이블된 훈련 샘플이 있으므로(기대 출력값, 즉 구역의 중간 주택 가격) 전형적인 지도 학습 작업
  - 모델이 값을 예측 – 회귀
    - 예측에 사용할 특성이 여러 개(구역의 인구, 중간 소득 등) - 다중 회귀(multiple regression)
    - 각 구역마다 하나의 값을 예측 - 단변량 회귀(univariate regression)
      - 구역마다 여러 개의 값을 예측한다면 다변량 회귀(multivariate regression)
  - 시스템으로 들어오는 데이터에 연속적인 흐름이 없으므로 빠르게 변하는 데이터에 적응하지 않아도 되고, 데이터가 메모리에 들어갈 만큼 충분히 작으므로 일반적인 배치 학습이 적절

2.2 큰 그림 보기(3)

2.2.2 성능 측정 지표 선택

- 회귀 문제의 전형적인 성능 지표는 평균 제곱근 오차(root mean square error, RMSE)
  - 오차가 커질수록 이 값은 더욱 커지므로 예측에 얼마나 많은 오차가 있는지 가늠하게 해줌
- 이상치로 보이는 구역이 많다면 평균 절대 오차(mean absolute error, MAE, 평균 절대 편차)를 고려
- RMSE와 MAE 모두 예측값의 벡터와 타깃값의 벡터 사이의 거리를 재는 방법
  - 거리 측정에는 여러 가지 방법(또는 노름(norm))을 사용
    - 제곱항을 합한 것의 제곱근(RMSE) 계산은 유클리디안 노름(Euclidean norm)에 해당
      - l_2노름 ($\|\cdot\|_2$ 또는 $\|\cdot\|$)
    - 절댓값의 합을 계산하는 것은 l_1노름에 해당하며 $\|\cdot\|$ 로 표기
      - 도시의 구획이 직각으로 나뉘어 있을 때 도시의 두 지점 사이의 거리를 측정하는 것과 같아 맨해튼 노름이라고도 함
    - 일반적으로 원소가 n개인 벡터 v의 l_k노름은 $\|v\|_k = (|v_0|^k + |v_1|^k + \cdots + |v_n|^k)^{\frac{1}{k}}$로 정의
      - l_0은 단순히 벡터에 있는 0이 아닌 원소의 수이고, l_∞는 벡터에서 가장 큰 절댓값

식 2-1 평균 제곱근 오차(RMSE)

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2}$$

식 2-2 평균 절대 오차(MAE)

$$\mathrm{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right|$$

2.2 큰 그림 보기(4)

2.2.3 가정 검사

- 지금까지 만든 가정을 나열하고 검사
  - 이 과정에서 심각한 문제를 일찍 발견할 수도 있음
  - 예) 시스템이 출력한 구역의 가격이 그대로 다음 머신러닝 시스템의 입력으로 사용될 거라 가정했는데, 하위 시스템에서 이 값을 ('저렴', '보통', '고가' 같은) 카테고리로 바꾸고 가격 대신 카테고리를 사용하면 어떻게 될까?
    - 정확한 가격을 구하는 것이 전혀 중요하지 않고, 올바른 카테고리를 구하는 시스템이 필요
    - 따라서, 회귀가 아니라 분류 작업

2.3 데이터 가져오기(1)

- 오픈 소스 예제
  - 주피터 노트북 코드 예제: https://github.com/rickiepark/handson-ml3
    - 주피터 노트북은 텍스트, 이미지, 실행 가능한 코드(이 책의 경우 파이썬)를 담은 인터랙티브한 문서
    - 온라인에서 주피터 노트북을 바로 실행할 수 있는 무료 서비스인 구글 코랩(Google Colab)으로 이 노트북을 실행

2.3 데이터 가져오기(2)

2.3.1 구글 코랩을 사용하여 예제 코드 실행하기

- 브라우저를 열고 http://bit.ly/3K5sOoj에 방문
  - 구글 코랩 페이지가 열리고 이 책의 주피터 노트북 목록이 보임
  - 장마다 하나의 노트북이 있고, 추가 노트북과 넘파이, 맷플롯립, 판다스, 선형대수, 미분을 위한 튜토리얼이 있음
    - 예) 02_end_to_end_machine_learning_project.ipynb를 클릭하면 2장의 노트북이 구글 코랩에서 열림
  - 하나의 주피터 노트북은 셀(cell)의 목록으로 구성되고, 각 셀은 실행 가능한 코드나 텍스트를 담고 있음
  - 새로운 코드 셀과 런타임 할당

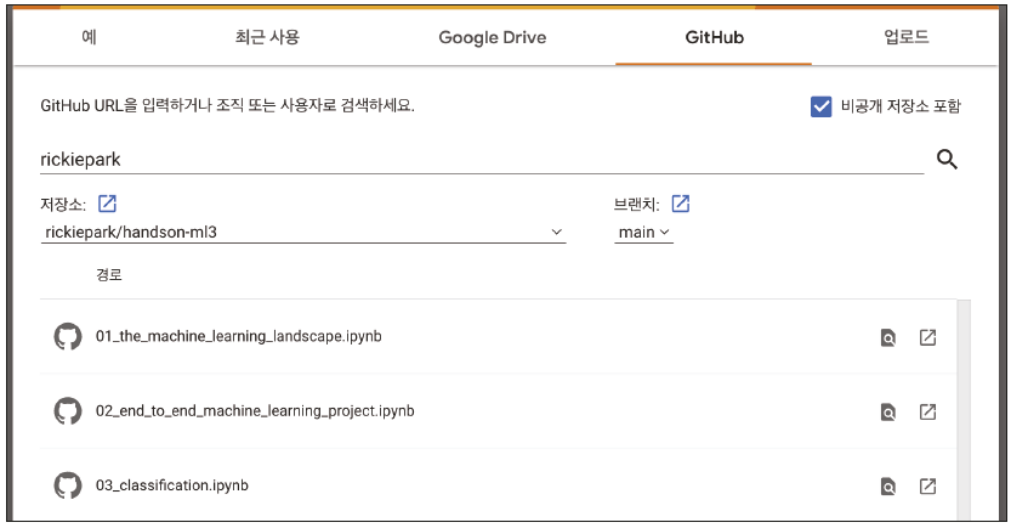

그림 2-3 구글 코랩에서 노트북 목록 보기

그림 2-4 구글 코랩에서 열린 노트북

2.3 데이터 가져오기(3)

2.3.2 코드와 데이터 저장하기

- 코랩 노트북에서 수정한 내용은 브라우저 탭이 열려있는 동안만 유지
  - 수정한 내용을 유지하려면 [파일] → [Drive에 사본 저장] 또는 [파일] → [다운로드] → [.ipynb 다운로드]

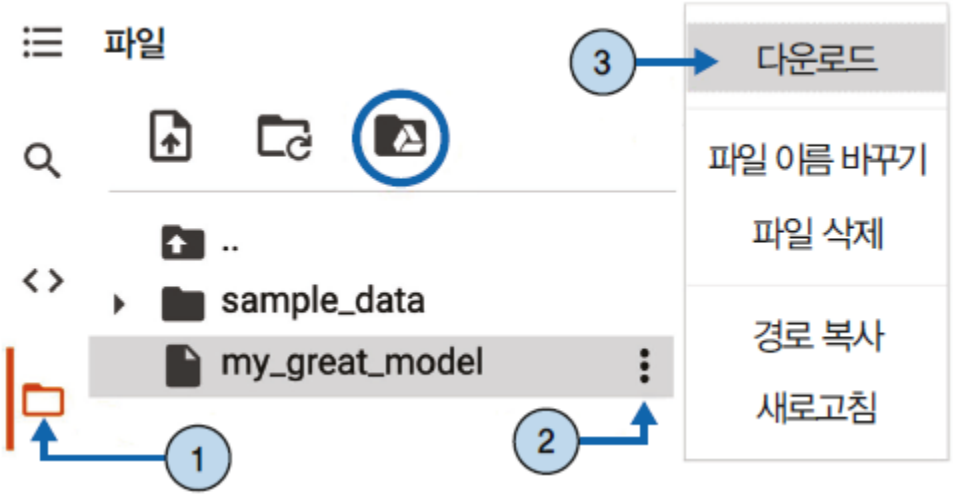

그림 2-5 구글 코랩 런타임에서 파일을 다운로드하거나(① → ② → ③), 구글 드라이브에 마운팅하기(① → 원 아이콘)

2.3 데이터 가져오기(4)

2.3.3 대화식 환경의 편리함과 위험

- 주피터 노트북은 인터랙티브한 환경
  - 각 셀을 하나씩 실행하다가 도중에 멈추고 새로운 셀을 추가한 다음 코드를 입력하여 실행 가능
  - 다시 돌아가서 이전 셀을 실행할 수도 있음
  - 잘못된 순서로 셀을 실행하기가 매우 쉬움
  - 어떤 셀을 실행하는 것을 잊어버릴 수 있음
    - 이런 경우에는 이어지는 코드 셀이 실행되지 않을 가능성이 높음
    - 각 노트북에 있는 첫 번째 코드 셀은 설정 코드(예: 패키지 임포트)를 담고 있기 때문에 가장 먼저 실행해야 함

2.3 데이터 가져오기(5)

2.3.4 책의 코드와 노트북의 코드

- 이따금 책에 있는 코드와 노트북에 있는 코드가 조금 다른 경우가 있음
  - 이 책을 읽고 있을 때 라이브러리가 조금 변경될 수 있음
    - 책에 있는 코드를 실행했을 때 에러가 발생했다면 노트북에 있는 코드를 참조
  - 노트북에는 그래프를 꾸미고(레이블 추가, 폰트 크기 설정 등) 책에 사용하기 위해 고해상도로 저장하는 코드가 추가
    - 이런 코드는 무시해도 괜찮음
  - 최대한 코드를 읽기 편하고 단순하게 작성
    - 함수와 클래스 정의는 매우 적으며 복잡하지 않게 구성
    - 실행하는 코드를 바로 보여주고 검색이 필요한 여러 추상화 계층에 감춰지지 않도록 하기 위함
    - 코드를 이리저리 수정하기도 쉬움
    - 단순함을 위하여 에러 처리는 제한적으로 수행

2.3 데이터 가져오기(6)

2.3.5 데이터 다운로드

- 모든 데이터가 들어 있는 CSVcomma-separated value 파일인 housing.csv를 압축한 housing.tgz 파일을 내려받기
  - 데이터를 내려받는 일을 자동화하면 여러 기기에 데이터셋을 설치해야 할 때도 편리
  - 데이터를 추출하고 로드하는 함수

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()
```

2.3 데이터 가져오기(7)

2.3.6 데이터 구조 훑어보기

- 데이터프레임의 head() 메서드를 사용해 데이터에 있는 처음 다섯 행을 확인
  - 각 행은 하나의 구역을 나타냄
  - 10 개의 특성
    1) longitude
    2) latitude
    3) housing_median_age
    4) total_rooms
    5) total_bedrooms, population
    6) households
    7) median_income
    8) median_
    9) house_value
    10) ocean_proximity

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

그림 2-6 데이터셋의 처음 다섯 행

2.3 데이터 가져오기(8)

- info() 메서드
  - 데이터에 관한 간략한 설명을 보여줌
  - 특히 전체 행 수, 각 특성의 데이터 타입과 널 null이 아닌 값의 개수를 확인하는 데 유용

```
>>> housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

- 20,640개의 샘플
- total_bedrooms 특성은 20,433개만 널값이 아님
  - 207개의 구역은 이 특성을 가지고 있지 않다는 의미
- ocean_proximity 필드만 빼고 모든 특성이 숫자형

2.3 데이터 가져오기(9)

- value_counts() 메서드
  - 각 카테고리마다 얼마나 많은 구역이 있는지는 확인

```
>>> housing["ocean_proximity"].value_counts()
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: ocean_proximity, dtype: int64
```

2.3 데이터 가져오기(10)

- describe() 메서드
  - 숫자형 특성의 요약 정보
  - std 행은 값이 퍼져 있는 정도를 측정하는 표준 편차
  - 25%, 50%, 75% 행은 백분위수(percentile)

```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

그림 2-7 숫자형 특성의 요약 정보

2.3 데이터 가져오기(11)

- hist() 메서드
  - 전체 데이터셋에 대해 모든 숫자형 특성에 대한 히스토그램을 출력

```
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(12, 8))
plt.show()
```

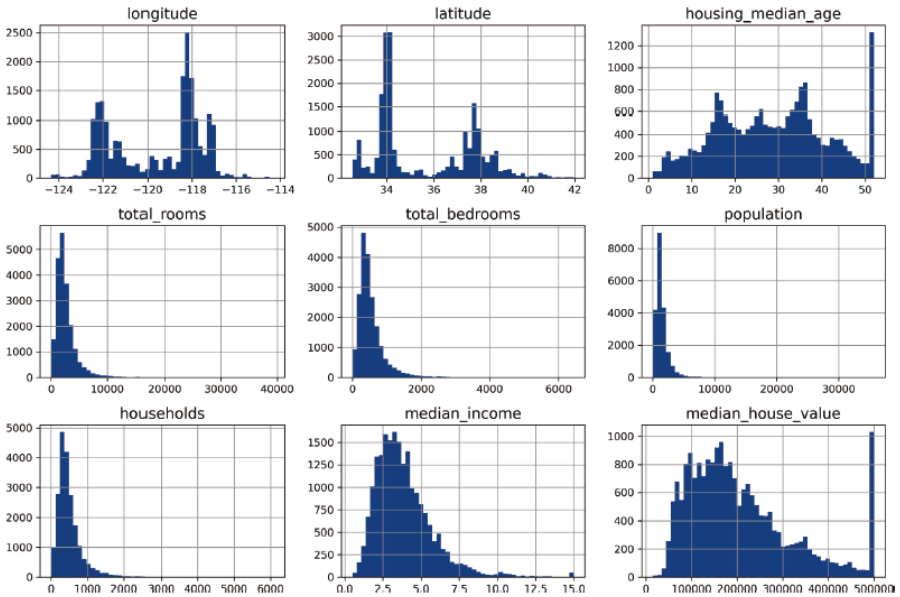

그림 2-8 모든 숫자형 특성에 대한 히스토그램

- 중간 소득(median_income) 특성이 US 달러로 표현되어 있지 않음
- 중간 주택 연도 (housing_median_age)와 중간 주택 가격 (median_house_value) 역시 최댓값과 최솟값을 한정
- 특성들의 스케일이 서로 많이 다름
- 많은 히스토그램에서 오른쪽 꼬리가 더 길게 나타남

2.3 데이터 가져오기(12)

2.3.7 테스트 세트 만들기

- 데이터 스누핑(data snooping) 편향
- 테스트 세트를 생성
  - 랜덤으로 어떤 샘플을 선택해서 데이터 셋의 20% 정도(데이터셋이 매우 크다면 그보다 적게)를 떼어놓음

```
import numpy as np

def shuffle_and_split_data(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

```
>>> train_set, test_set = shuffle_and_split_data(housing, 0.2)
>>> len(train_set)
16512
>>> len(test_set)
4128
```

2.3 데이터 가져오기(13)

- 프로그램을 다시 실행하면 다른 테스트 세트가 생성
  - 처음 실행에서 테스트 세트를 저장하고 다음번 실행에서 이를 불러들임
  - 다른 방법은 항상 같은 난수 인덱스가 생성되도록 np.random.permutation()을 호출하기 전에 난수 발생기의 초깃값을 지정
  - 위의 두 해법 모두 다음번에 업데이트된 데이터셋을 사용할 때 문제
- 데이터셋을 업데이트한 후에도 안정적인 훈련/테스트 분할을 가능하게 하는 일반적인 해결책
  - 샘플의 식별자를 사용하여 테스트 세트로 보낼지 말지 결정

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

```
housing_with_id = housing.reset_index()   # index 열이 추가된 데이터프레임이
                                          # 반환됩니다.
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

```
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

2.3 데이터 가져오기(14)

- train_test_split 함수
  - 데이터셋을 여러 서브셋(subset)으로 나누기
  - shuffle_and_split_data() 함수와 아주 비슷하지만 두 가지 추가되는 특성
    1) 난수 초깃값을 지정할 수 있는 random_state 매개변수
    2) 행의 개수가 같은 여러 개의 데이터셋을 넘겨서 동일한 인덱스를 기반으로 나누기

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

2.3 데이터 가져오기(15)

- 계층적 샘플링(stratified sampling)
- 테스트 세트가 전체 데이터셋에 있는 여러 소득 카테고리를 잘 대표하도록 작성
  - 중간 소득이 연속적인 숫자형 특성이므로 소득에 대한 카테고리 특성을 생성
- pd.cut() 함수
  - 카테고리 5개를 가진 소득 카테고리 특성을 생성

```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid=True)
plt.xlabel("소득 카테고리")
plt.ylabel("구역 개수")
plt.show()
```

2.3 데이터 가져오기(16)

- sklearn.model_selection 패키지의 여러 가지 분할기(splitter) 클래스
- split() 메서드
  - 훈련과 테스트 분할에 대한 반복자를 반환
  - 훈련과 테스트 데이터 자체가 아니라 인덱스를 반환

```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing, housing["income_cat"]):
    strat_train_set_n = housing.iloc[train_index]
    strat_test_set_n = housing.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

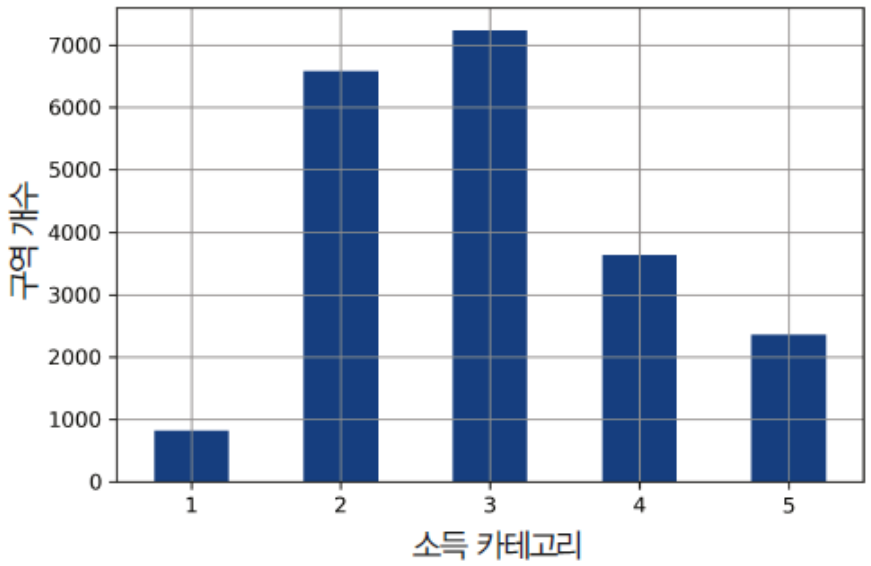

그림 2-9 소득 카테고리의 히스토그램

2.3 데이터 가져오기(17)

- 첫 번째 분할

```
strat_train_set, strat_test_set = strat_splits[0]
```

- 하나의 분할이 필요한 경우 train_test_split() 함수와 stratify 매개변수를 사용

```
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```

2.3 데이터 가져오기(18)

- 테스트 세트의 소득 카테고리 비율
  - 비슷한 코드로 전체 데이터셋에 있는 소득 카테고리의 비율을 측정

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
3    0.350533
2    0.318798
4    0.176357
5    0.114583
1    0.039729
Name: income_cat, dtype: float64
```

소득 카테고리	전체 %	계층 샘플링 %	랜덤 샘플링 %	계층 샘플링 오차 %	랜덤 샘플링 오차 %
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

그림 2-10 계층 샘플링과 순수한 랜덤 샘플링의 샘플링 편향 비교

- 계층 샘플링을 사용해 만든 테스트 세트는 전체 데이터셋에 있는 소득 카테고리의 비율과 거의 같음
- 반면 일반 랜덤 샘플링으로 만든 테스트 세트는 비율이 많이 달라짐

2.3 데이터 가져오기(19)

- 예제에서는 income_cat 특성을 다시 사용하지 않으므로 이 열을 삭제하고 데이터를 원래 상태로 되돌림
  - 판다스 데이터프레임의 drop 메서드는 행 또는 열을 삭제
  - axis 매개변수의 기본값이 0일 때는 행을 삭제하고 1일 때는 열을 삭제
  - 기본값이 False인 inplace 매개변수를 True로 설정하면 호출된 객체에 새로운 데이터프레임을 재할당하고 아무런 값도 반환하지 않음

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

2.4 데이터 이해를 위한 탐색과 시각화(1)

- 테스트 세트를 떼어놓았는지 확인하고 훈련 세트에 대해서만 탐색
  - 훈련 세트가 매우 크다면 탐색 단계에서 조작을 간단하고 빠르게 하기 위해 탐색을 위한 세트를 별도로 샘플링
  - 학습 예제에서는 훈련 세트의 크기가 작으므로 훈련 세트 전체를 사용

- 전체 훈련 세트에 대해서 다양한 변환을 실험하기 때문에 나중에 되돌릴 수 있도록 원본 데이터의 복사본을 생성

```
housing = strat_train_set.copy()
```

2.4 데이터 이해를 위한 탐색과 시각화(2)

2.4.1 지리적 데이터 시각화하기

- 모든 구역을 산점도로 만들어 데이터 시각화
  - 데이터셋에는 지리 정보(위도와 경도)가 포함

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.xlabel("경도")
plt.ylabel("위도")
plt.show()
```

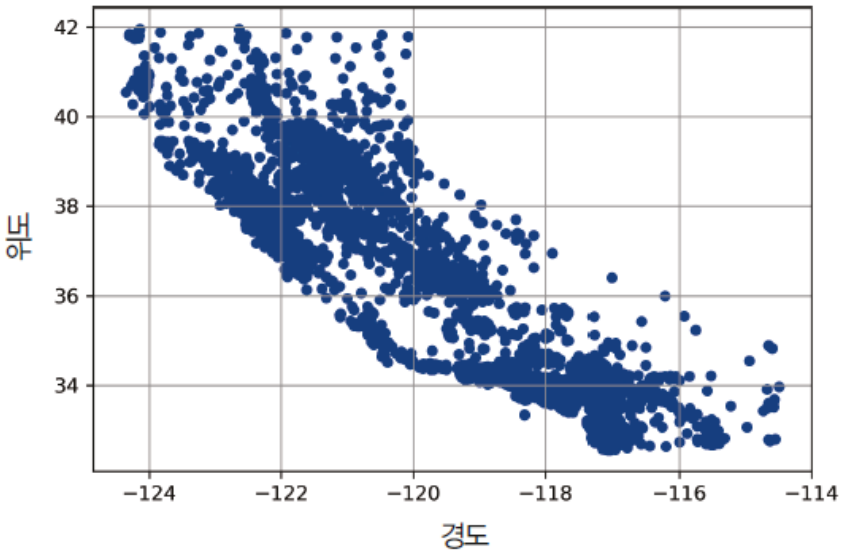

그림 2-11 데이터의 지리적인 산점도

2.4 데이터 이해를 위한 탐색과 시각화(3)

- [그림 2-11]에 alpha 옵션을 0.2로 추가하여 데이터 포인트가 밀집된 영역을 강조

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
plt.xlabel("경도")
plt.ylabel("위도")
plt.show()
```

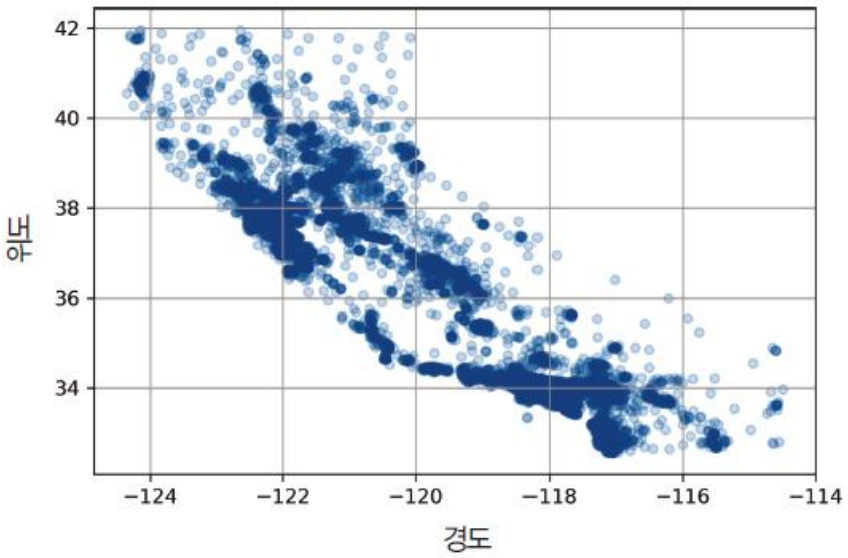

그림 2-12 밀집된 지역이 잘 부각된 산점도

2.4 데이터 이해를 위한 탐색과 시각화(4)

- 주택 가격 시각화
  - 원의 반지름은 구역의 인구(매개변수 s), 색상은 가격(매개변수 c)
  - 컬러 맵(color map) - 파란색(낮은 가격)에서 빨간색(높은 가격)까지의 범위를 가지는 jet을 사용(매개변수 cmap)

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             cax=plt.gcf().get_axes()[1]
             cax.set_ylabel("중간 주택 가격")
             plt.xlabel("경도")
             plt.ylabel("위도")
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

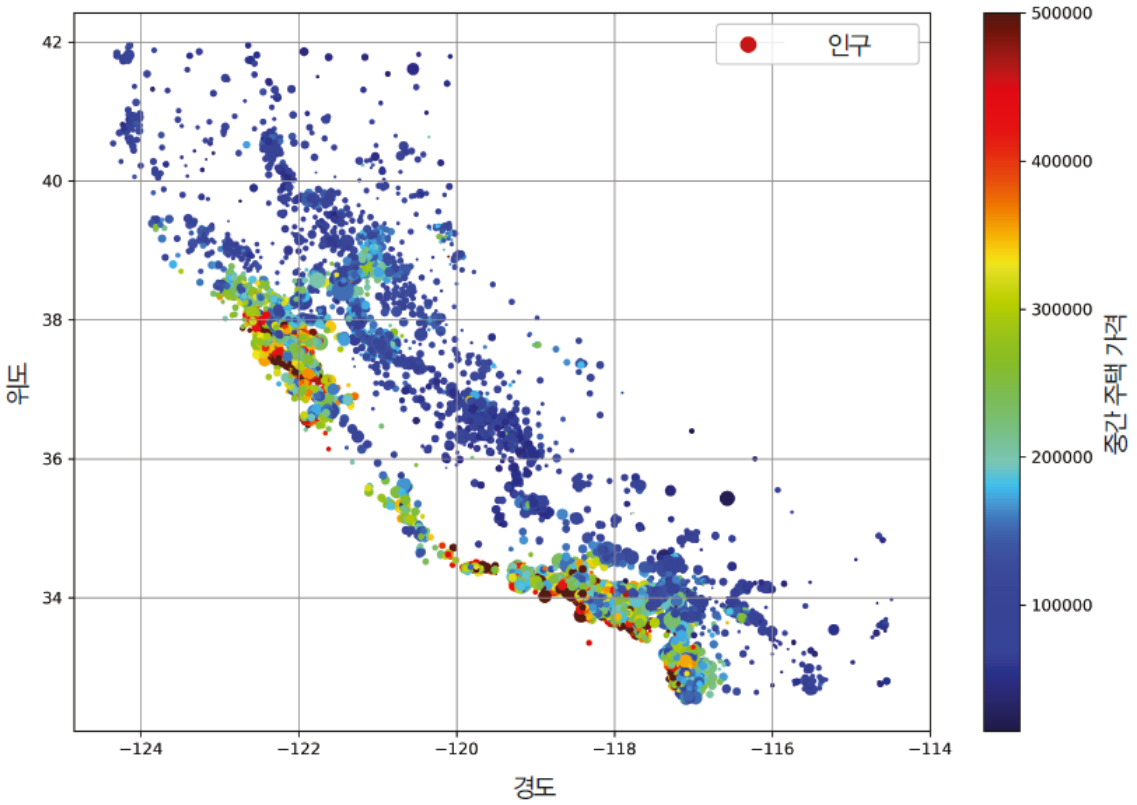

그림 2-13 캘리포니아 주택 가격: 빨간색은 높은 가격, 파란색은 낮은 가격, 큰 원은 인구가 밀집된 지역을 나타냄

2.4 데이터 이해를 위한 탐색과 시각화(5)

2.4.2 상관관계 조사하기

- corr() 메서드
  - 특성 간의 표준 상관계수 standard correlation coefficient (피어슨 Pearson의 r)를 계산

```
corr_matrix = housing.corr(numeric_only=True)
```

  - 중간 주택 가격과 다른 특성 사이의 상관관계 크기

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
Name: median_house_value, dtype: float64
```

2.4 데이터 이해를 위한 탐색과 시각화(6)

- 판다스의 scatter_matrix 함수
  - 특성 사이의 상관관계를 확인하기 위해 숫자형 특성 간 산점도 시각화
  - 숫자형 특성이 11개이므로 총 11^2=121개의 그래프가 그려짐

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
plt.show()
```

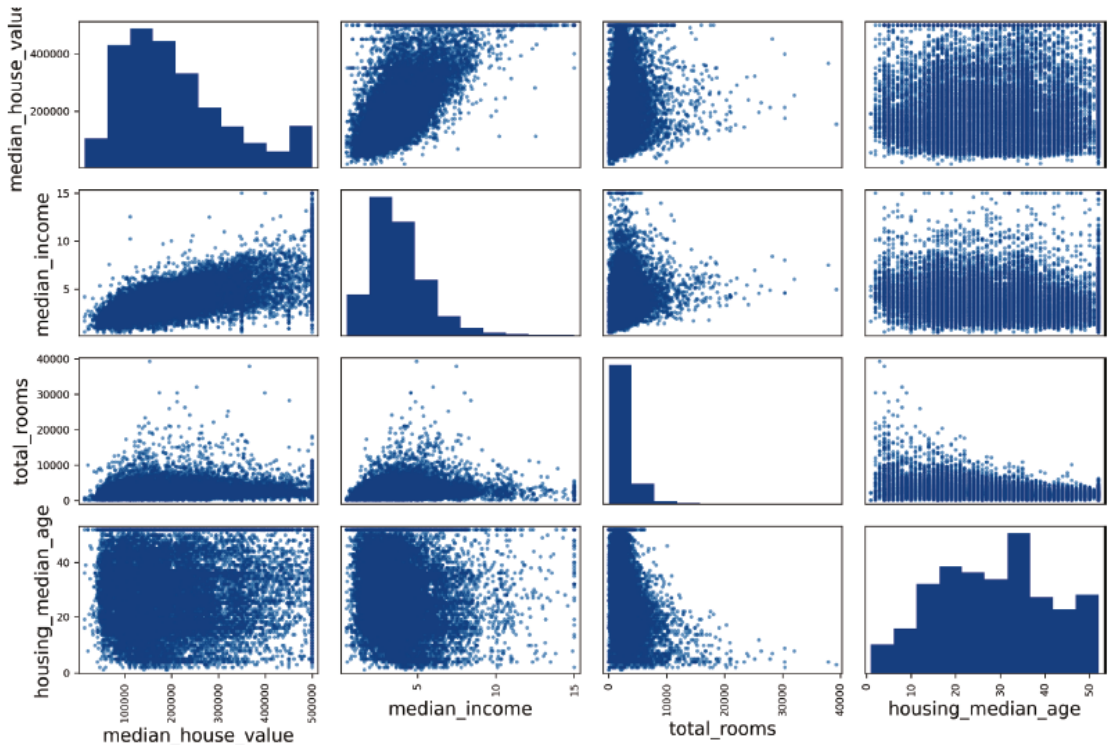

그림 2-14 이 산점도 행렬은 수치형 특성 간의 산점도와 각 수치형 특성의 히스토그램을 출력

2.4 데이터 이해를 위한 탐색과 시각화(7)

- 중간 소득(median_income)으로 중간 주택 가격(median_house_value)을 예측하고 산점도를 확대

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
             plt.xlabel("중간 소득")
             plt.ylabel("중간 주택 가격")
plt.show()
```

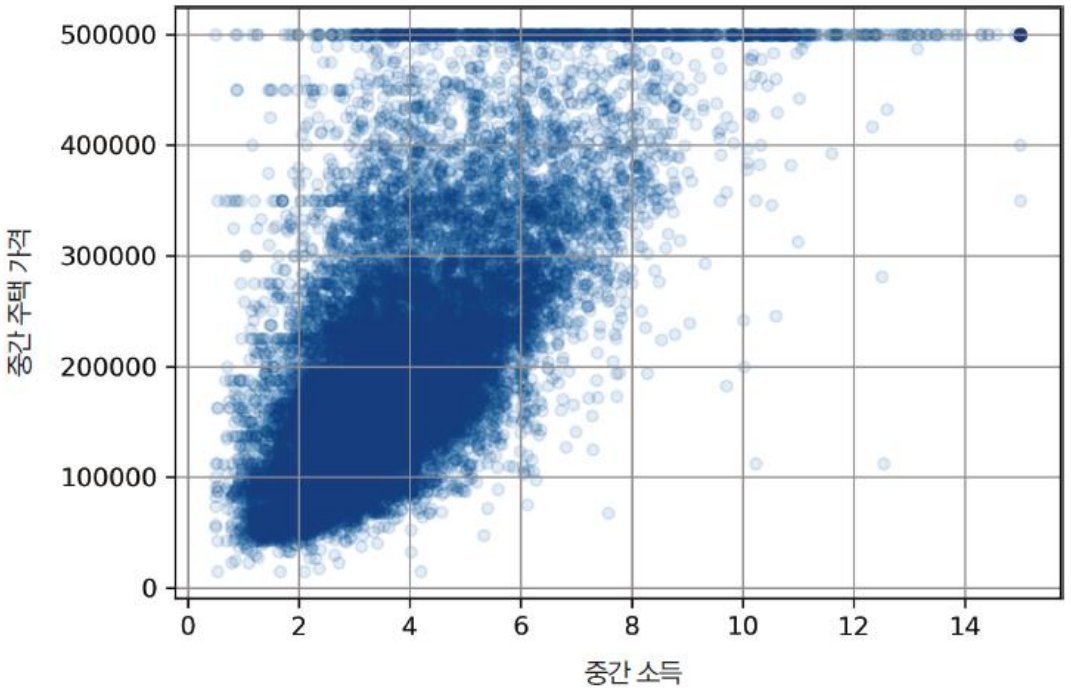

그림 2-15 중간 소득 vs. 중간 주택 가격

2.4 데이터 이해를 위한 탐색과 시각화(8)

! CAUTION

- 상관계수는 선형적인 상관관계만 측정(x가 증가하면 y는 증가 또는 감소하는 경우)
  - 비선형적인 관계는 잡을 수 없음(예 x가 0에 가까워지면 y가 증가하는 경우)
  - [그림 2-16]은 다양한 데이터셋과 상관계수
    - 마지막 줄에 있는 그래프들은 두 축이 완전히 독립적이지 않음에도 상관계수가 0인 비선형 관계
    - 두 번째 줄은 상관계수가 1이거나 -1인 경우
    - 상관계수는 기울기와 상관없음
      - 예) 인치(inch) 단위의 키는 피트(feet)나 나노미터(nanometer) 단위의 키와 상관계수가 1

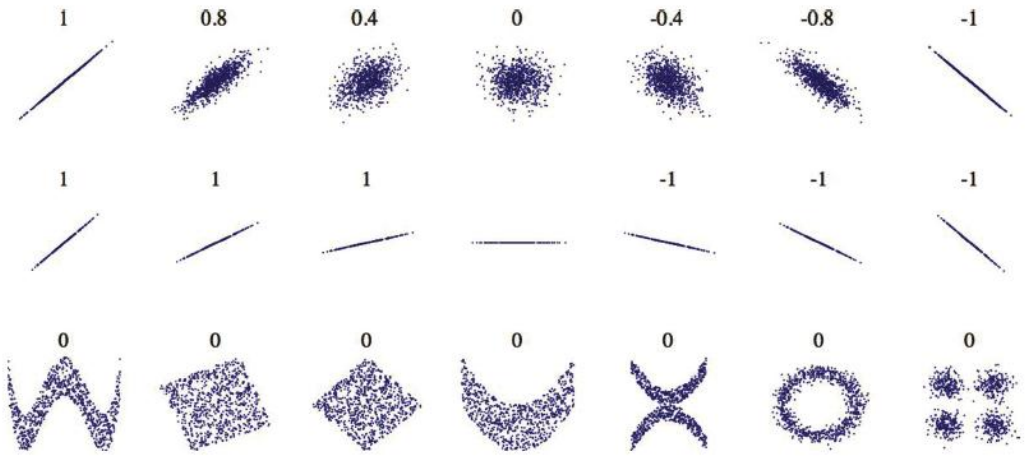

그림 2-16 여러 가지 데이터셋에 나타난 표준 상관계수

2.4 데이터 이해를 위한 탐색과 시각화(9)

2.4.3 특성 조합으로 실험하기

- 특성을 여러 가지로 조합하고 상관관계 확인
  - 방 개수 vs. 가구 수
  - 침실 개수 vs. 방 개수
  - 가구당 인원 vs. 가구 수

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["population_per_house"]=housing["population"] / housing["households"]
```

```
>>> corr_matrix = housing.corr(numeric_only=True)
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value          1.000000
median_income               0.688380
rooms_per_house             0.143663
total_rooms                 0.137455
housing_median_age          0.102175
households                  0.071426
total_bedrooms              0.054635
population                 -0.020153
people_per_house           -0.038224
longitude                  -0.050859
latitude                   -0.139584
bedrooms_ratio             -0.256397
Name: median_house_value, dtype: float64
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(1)

- 데이터 자동화의 장점
  - 어떤 데이터셋에 대해서도 데이터 변환을 손쉽게 반복(다음번에 새로운 데이터셋을 사용할 때 활용)
  - 향후 프로젝트에 재사용 가능한 변환 라이브러리를 점진적으로 구축
  - 실제 시스템에서 알고리즘에 새 데이터를 주입하기 전에 이 함수를 사용해 변환
  - 여러 가지 데이터 변환을 쉽게 시도해볼 수 있고 어떤 조합이 가장 좋은지 확인

- 원래 훈련 세트로 복원(strat_train_set을 다시 복사)
  - 예측 변수와 타깃값에 같은 변형을 적용하지 않기 위해 예측 변수와 레이블을 분리
  - drop()은 기본적으로 데이터 복사본을 만들어 반환하며 strat_train_set에는 영향을 주지 않음

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(2)

2.5.1 데이터 정제

- 머신러닝 알고리즘은 누락된 특성을 다루지 못하므로 이를 처리할 수 있는 함수를 몇 개 생성
  - total_bedrooms 특성에 값이 없는 경우
    1) 해당 구역을 제거
    2) 전체 특성을 삭제
    3) 누락된 값을 어떤 값으로 채우기(0, 평균, 중간값 등) - 이를 대체(imputation)라고 함

  - 판다스 데이터프레임의 dropna(), drop(), fillna() 메서드를 이용

```
housing.dropna(subset=["total_bedrooms"], inplace=True)   # 옵션 1

housing.drop("total_bedrooms", axis=1, inplace=True)      # 옵션 2

median = housing["total_bedrooms"].median()               # 옵션 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(3)

- SimpleImputer 클래스
  - 각 특성의 중간값을 저장하고 있어 유용
  - 훈련 세트뿐만 아니라 검증 세트와 테스트 세트 그리고 모델에 주입될 새로운 데이터에 있는 누락된 값을 대체

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

- 중간값은 수치형 특성에서만 계산될 수 있기 때문에 수치 특성만 가진 데이터 복사본을 생성
  - 텍스트 특성인 ocean_proximity가 제외

```
housing_num = housing.select_dtypes(include=[np.number])
```

- imputer 객체의 fit() 메서드를 사용해 훈련 데이터에 적용

```
imputer.fit(housing_num)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(4)

- imputer는 각 특성의 중간값을 계산해서 그 결과를 객체의 statistics_ 속성에 저장

```
>>> imputer.statistics_
array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
>>> housing_num.median().values
array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
```

- 학습된 imputer 객체를 사용해 훈련 세트에서 누락된 값을 학습한 중간값으로 변경

```
X = imputer.transform(housing_num)
```

- 누락된 값을 평균(strategy="mean") 이나 가장 자주 등장하는 값(strategy="most_frequent") 또는 상수(strategy="constant", fill_value=...)로 바꿀 수도 있음
  - 마지막 두 방법은 수치가 아닌 데이터를 지원
- imputer.transform(housing_num)의 출력은 넘파이 배열
  - X를 데이터프레임으로 감싸서 housing_num으로부터 열 이름과 인덱스를 복원

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(5)

사이킷런의 설계 철학

- 일관성: 모든 객체는 일관되고 단순한 인터페이스를 공유
  - 추정기(estimator) : 데이터셋을 기반으로 일련의 모델 파라미터들을 추정하는 객체
  - 변환기(transformer) : (imputer 같이) 데이터셋을 변환하는 추정기
  - 예측기(predictor) : 주어진 데이터셋에 대해 예측을 만드는 추정기
- 검사 가능
- 클래스 남용 방지
- 조합성: 기존의 구성 요소를 최대한 재사용
- 합리적인 기본값

2.5 머신러닝 알고리즘을 위한 데이터 준비(6)

2.5.2 텍스트와 범주형 특성 다루기

- 학습 데이터셋의 ocean_proximity에서 특성값 확인

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(8)
       ocean_proximity
13096         NEAR BAY
14973        <1H OCEAN
3785            INLAND
14689           INLAND
20507       NEAR OCEAN
1286            INLAND
18078        <1H OCEAN
4396          NEAR BAY
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(7)

- OrdinalEncoder 클래스
  - 카테고리를 텍스트에서 숫자로 변환

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

  - housing_cat_encoded에 인코딩된 몇 개의 값을 확인

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(8)

- categories_ 인스턴스 변수
  - 범주형 특성마다 1D 카테고리 배열을 담은 리스트가 반환

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

- 머신러닝 알고리즘이 가까이 있는 두 값을 떨어져 있는 두 값보다 더 비슷하다고 생각
  - 일부 경우에는 괜찮음(예: 'bad', 'average', 'good', 'excellent'와 같이 순서가 있는 카테고리의 경우)
  - ocean_proximity 열에는 해당되지 않음
    (예를 들어 카테고리 0과 1보다 카테고리 0과 4가 확실히 더 비슷)
- 일반적으로 카테고리별 이진 특성을 만들어 해결
  - 카테고리가 '<1H OCEAN ' 일 때 한 특성이 1이고(그 외 특성은 0), 카테고리가 'INLAND'일 때 다른 한 특성이 1이 되는(역시 그 외에는 0) 방식
- 원-핫 인코딩(one-hot encoding)
  - 한 특성만 1이고(핫) 나머지는 0
- 더미(dummy) 특성

2.5 머신러닝 알고리즘을 위한 데이터 준비(9)

- OneHotEncoder 클래스
  - 범주의 값을 원-핫 벡터로 바꾸기

```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

  - 기본적으로 OneHotEncoder의 출력은 넘파이 배열이 아니라 사이파이 희소 행렬
    - 희소 행렬은 0이 대부분인 행렬을 매우 효율적으로 표현
    - 내부적으로 0이 아닌 값과 그 위치만 저장

```
>>> housing_cat_1hot
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
 with 16512 stored elements in Compressed Sparse Row format>
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(10)

- toarray() 메서드
  - 대부분 희소행렬을 보통의 2D 배열처럼 사용할 수 있지만 (밀집된) 넘파이 배열로 반환

```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

- OneHotEncoder를 만들 때 sparse=False(또는 sparse_output=False)로 설정하여 transform() 메서드가 일반적인 넘파이 배열을 반환하게 할 수 있음

2.5 머신러닝 알고리즘을 위한 데이터 준비(11)

- OrdinalEncoder처럼 인코더의 categories_ 인스턴스 변수를 사용

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

- get_dummies() 함수
  - 범주형 특성을 원-핫 표현으로 바꿔서 카테고리마다 하나의 이진 특성을 생성

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
>>> pd.get_dummies(df_test)
   ocean_proximity_INLAND ocean_proximity_NEAR BAY
0                       1                       0
1                       0                       1
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(12)

- cat_encoder를 (fit_transform()이 아니라 transform()으로) df_test에 적용했을 때 출력
  - get_dummies()는 두 개의 카테고리만 보았기 때문에 두 개 열만 출력
  - 반면 OneHotEncoder는 학습된 카테고리마다 하나의 열을 순서대로 출력

```
>>> cat_encoder.transform(df_test)
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

  - 알 수 없는 카테고리(예: "<2H OCEAN")를 담은 데이터프레임을 get_dummies()에 주입하면 아무런 문제없이 변환된 결과를 출력

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN", "ISLAND"]})
>>> pd.get_dummies(df_test_unknown)
   ocean_proximity_<2H OCEAN  ocean_proximity_ISLAND
0                          1                       0
1                          0                       1
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(13)

- OneHotEncode는 알 수 없는 카테고리를 감지하고 예외를 발생시킴
  - handle_unkown 매개변수를 "ignore"로 지정하여 알 수 없는 카테고리를 그냥 0으로 나타낼 수 있음

```
>>> cat_encoder.handle_unknown = "ignore"
>>> cat_encoder.transform(df_test_unknown)
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

  - 데이터프레임을 사용하여 사이킷런 추정기를 훈련할 때 추정기는 열 이름을 feature_names_in_ 속성에 저장
  - 사이킷런은 (transform()이나 predict()를 통해) 이 추정기에 입력된 모든 데이터프레임이 동일한 열 이름을 갖는지 확인
  - 변환기는 출력으로 데이터프레임을 만들 수 있도록 get_feature_names_out() 메서드도 제공

```
>>> cat_encoder.feature_names_in_
array(['ocean_proximity'], dtype=object)
>>> cat_encoder.get_feature_names_out()
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)
>>> df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
...                          columns=cat_encoder.get_feature_names_out(),
...                          index=df_test_unknown.index)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(14)

2.5.3 특성 스케일과 변환

- 특성 스케일링(feature scaling)
- 모든 특성의 범위를 같게 만들기
  - min-max 스케일링 – 정규화(normalization)
    - MinMaxScaler 변환기와 feature_range 매개변수를 사용

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

  - 표준화(standardization)
    - StandardScaler 변환기

```
from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(15)

- 멱법칙 분포(power law distribution)
  - 특성 분포의 꼬리가 아주 길고 두껍다면 특성을 로그값으로 바꾸는 것이 도움
    - 예) population 특성

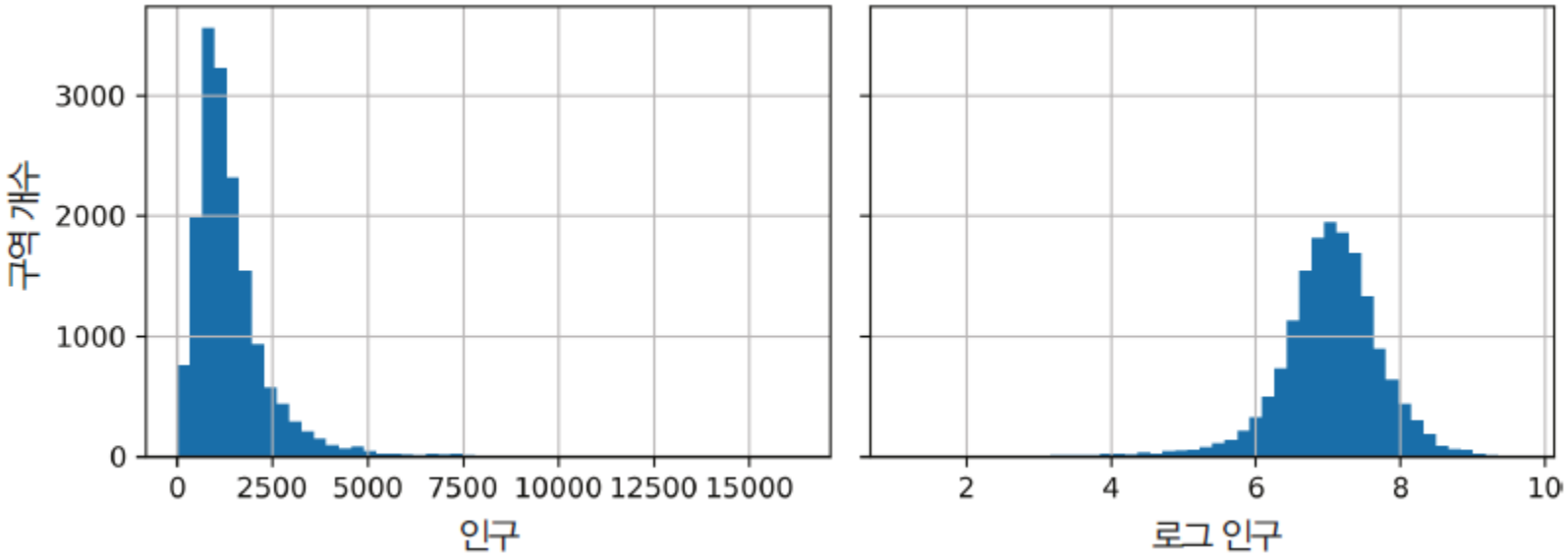

그림 2-17 특성을 가우스 분포에 가깝게 변환하기

2.5 머신러닝 알고리즘을 위한 데이터 준비(16)

- 버킷타이징(bucketizing)
  - 거의 동일한 크기의 버킷을 사용하면 거의 균등 분포인 특성을 만들 수 있음
- 멀티모달 분포(multimodal distribution) 변환
  - 버킷 인덱스를 (예를 들어 OneHot Encoder을 사용하여) 인코딩
  - 중간 주택 연도와 특정 모드 사이의 유사도를 나타내는 특성을 추가
    - 유사도 측정은 일반적으로 입력값과 고정 포인트 사이의 거리에만 의존하는 방사 기저 함수(RBF)를 사용

2.5 머신러닝 알고리즘을 위한 데이터 준비(17)

- 방사 기저 함수(RBF)
  - 가장 널리 사용되는 RBF는 입력값이 고정 포인트에서 멀어질수록 출력값이 지수적으로 감소하는 가우스 RBF
  - rbf_kernel() 함수를 사용하여 중간 주택 연도와 35 사이의 유사도를 재는 새로운 가우스 RBF 특성을 생성

```
from sklearn.metrics.pairwise import rbf_kernel

age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

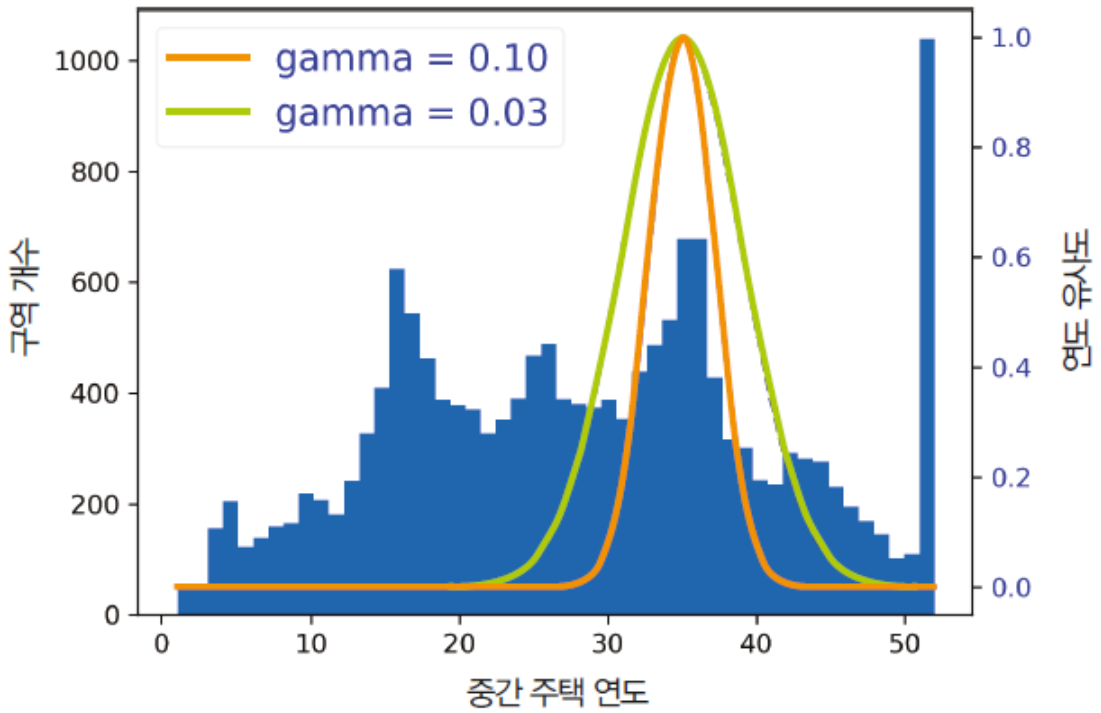

그림 2-18 중간 주택 가격과 35 사이의 유사도를 재는 가우스 RBF 특성

2.5 머신러닝 알고리즘을 위한 데이터 준비(18)

- 역변환을 수행하는 inverse_transform() 메서드

```
from sklearn.linear_model import LinearRegression
target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())
model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5]  # 새로운 데이터라고 가정합니다.
scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)
```

- TransformedTargetRegressor를 사용

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(19)

2.5.4 사용자 정의 변환기

- 로그 변환기를 만들어 population 특성에 적용
  - inverse_func 매개변수는 선택 사항

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

- 가우스 RBF 유사도를 계산하는 변환기 만들기

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                      kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(20)

- 각 구역과 샌프란시스코 사이의 지리적 유사도를 측정하는 특성을 추가

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords], gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

- 사용자 정의 변환기로 특성을 합치기
  - 첫 번째 입력 특성과 두 번째 특성 사이의 비율을 계산하는 FunctionTransformer

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
>>> ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]))

array([[0.5 ],
       [0.75]])
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(21)

- fit(), transform(), fit_transform()

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True): # *args나 **kwargs를 사용하지 않습니다!
        self.with_mean = with_mean

    def fit(self, X, y=None): # 사용하지 않더라도 y를 넣어야 합니다.
        X = check_array(X)    # X가 부동소수점 배열인지 확인합니다.
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1] # 모든 추정기는 fit()에서 이를 저장합니다.
        return self                      # 항상 self를 반환합니다!

    def transform(self, X):
        check_is_fitted(self) # (훈련으로) 학습된 속성이 있는지 확인합니다.
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
           X = X - self.mean_
        return X / self.scale_
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(22)

- 하나의 사용자 변환기가 구현 안에서 다른 추정기를 사용
  - fit() 메서드 안에서 훈련 데이터에 있는 핵심 클러스터를 식별하기 위해 KMeans 클래스를 사용하는 사용자 변환기
  - transform() 메서드에서 rbf_kernel()을 사용해 각 샘플이 클러스터 중심과 얼마나 유사한지 측정

```python
from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self # 항상 self를 반환합니다!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"클러스터 {i} 유사도" for i in range(self.n_clusters)]
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(23)

- k-평균은 데이터에 있는 클러스터를 찾는 군집 알고리즘
  - KMeans의 fit() 메서드는 선택적 매개변수 sample_weight를 제공하며 이 매개변수에 샘플의 상대적인 가중치를 지정
    - 결과를 동일하게 재현하려면 random_state 매개변수를 지정

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]],
                                           sample_weight=housing_labels)
```

  - k-평균으로 클러스터를 찾고, 각 구역과 10개의 클러스터 중심 사이의 가우스 RBF 유사도를 측정

```
>>> similarities[:3].round(2)
array([[0. , 0.14, 0. , 0. , 0. , 0.08, 0. , 0.99, 0. , 0.6 ],
       [0.63, 0. , 0.99, 0. , 0. , 0. , 0.04, 0. , 0.11, 0. ],
       [0. , 0.29, 0. , 0. , 0.01, 0.44, 0. , 0.7 , 0. , 0.3 ]])
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(24)

- [그림 2-19] k-평균으로 찾은 클러스터 10개의 중심
  - 구역의 색은 가장 가까운 클러스터 중심과의 지리적 유사도에 따라서 지정

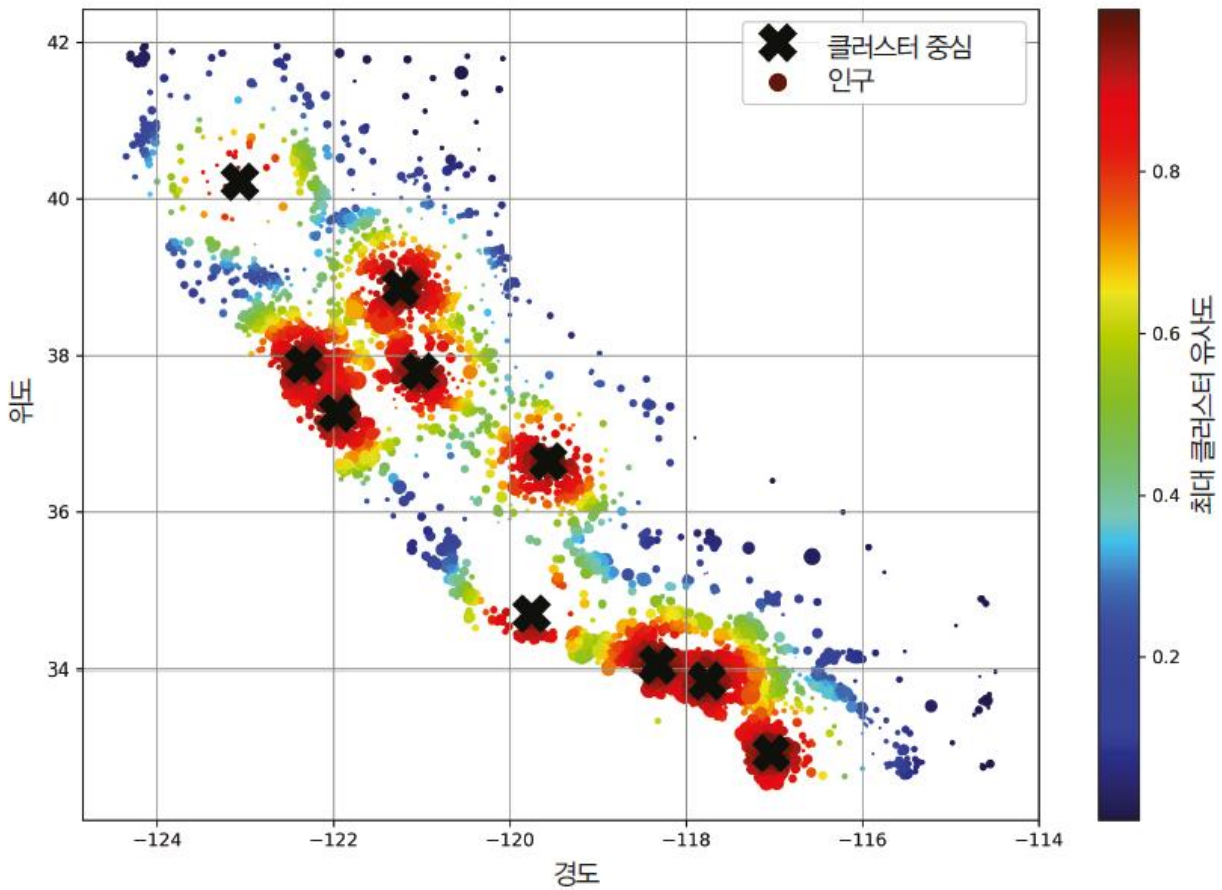

그림 2-19 가장 가까운 클러스터 중심과의 가우스 RBF 유사도

2.5 머신러닝 알고리즘을 위한 데이터 준비(25)

2.5.5 변환 파이프라인

- Pipeline 클래스
  - 변환을 순서대로 처리하도록 도와주는 기능
  - 수치 특성에서 누락된 값을 대체하고 스케일을 조정하는 간단한 파이프라인 코드

```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

  - make_pipeline() 함수
    - 위치 매개변수로 변환기를 받고 변환기의 클래스 이름을 밑줄 문자 없이 소문자로 바꾸어서 Pipeline 객체를 생성

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(26)

- 추정기가 변환기가 아니라 예측기라면 transform() 메서드 대신 predict() 메서드
  - 파이프라인의 fit_transform() 메서드를 호출하고 출력에서 처음 두 행의 출력을 소수점 둘째 자리에서 반올림

```
>>> housing_num_prepared = num_pipeline.fit_transform(housing_num)
>>> housing_num_prepared[:2].round(2)
array([[-1.42,  1.01, 1.86,  0.31,  1.37,  0.14,  1.39, -0.94],
       [ 0.6 , -0.7 , 0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

- 데이터프레임으로 재구성하는 get_feature_names_out() 메서드

```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(27)

- ColumnTransformer
  - 하나의 변환기로 각 열마다 적절한 변환을 적용하여 모든 열을 처리
  - ColumnTransformer는 (앞서 만든) num_pipeline을 수치형 특성에, cat_pipeline을 범주형 특성에 적용하는 코드

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
               "total_bedrooms", "population", "households", "median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(28)

- make_column_selector 클래스
  - 수치형이나 범주형처럼 주어진 타입의 모든 특성을 자동으로 선택
  - 자동으로 이름을 지정하는 make_column_transformer() 함수

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

- ColumnTransformer를 주택 데이터셋에 적용할 준비 완료

```
housing_prepared = preprocessing.fit_transform(housing)
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(29)

- 지금까지 실험한 모든 변환을 수행할 단일 파이프라인 만들기
  - 대부분의 머신러닝 알고리즘은 누락된 값을 기대하지 않기 때문에 수치형 특성의 경우 누락된 값을 중간값으로 대체
    범주형 특성의 경우 누락된 값을 가장 많이 등장하는 카테고리로 변경
  - 대부분의 머신러닝 알고리즘은 수치 입력만 받기 때문에 범주형 특성을 원-핫 인코딩
  - 비율 특성인 bedrooms_ratio, rooms_per_house, people_per_house를 계산하여 추가
    이런 특성은 중간 주택 가격과 상관관계가 높으므로 머신러닝 모델에 도움
  - 몇 가지 클러스터 유사도 특성을 추가
    위도와 경도보다 모델에 더 유용할 가능성이 높음
  - 대부분의 모델은 균등 분포나 가우스 분포에 가까운 특성을 선호하므로 꼬리가 두꺼운 분포를 띠는 특성을 로그값으로 변경
  - 대부분의 머신러닝 알고리즘은 모든 특성이 대체로 동일한 스케일을 가질 때 잘 작동하므로 모든 수치 특성을 표준화

2.5 머신러닝 알고리즘을 위한 데이터 준비(30)

- 모든 작업을 수행하는 파이프라인을 만드는 코드

```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # get_feature_names_out에 사용

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())
preprocessing = ColumnTransformer([
        ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
        ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
        ("people_per_house", ratio_pipeline(), ["population", "households"]),
        ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                               "households", "median_income"]),
        ("geo", cluster_simil, ["latitude", "longitude"]),
        ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
    ],
    remainder=default_num_pipeline) # 남은 특성: housing_median_age
```

2.5 머신러닝 알고리즘을 위한 데이터 준비(31)

- ColumnTransformer를 실행하면 모든 변환이 수행되고 24개의 특성을 가진 넘파이 배열이 출력

```
>>> housing_prepared = preprocessing.fit_transform(housing)
>>> housing_prepared.shape
(16512, 24)
>>> preprocessing.get_feature_names_out()
array(['bedrooms__ratio', 'rooms_per_house__ratio',
       'people_per_house__ratio', 'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity', [...],
       'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H OCEAN',
       'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
       'remainder__housing_median_age'], dtype=object)
```

2.6 모델 선택과 훈련(1)

2.6.1 훈련 세트에서 훈련하고 평가하기

- 선형 회귀 모델 훈련

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

- 훈련 세트에 적용하고 처음 다섯 개 예측과 레이블을 비교

```
>>> housing_predictions = lin_reg.predict(housing)
>>> housing_predictions[:5].round(-2) # -2 = 십의 자리에서 반올림
array([243700., 372400., 128800., 94400., 328300.])
>>> housing_labels.iloc[:5].values
array([458300., 483800., 101700., 96100., 361800.])
```

2.6 모델 선택과 훈련(2)

- 전체 훈련 세트에 대해서 이 회귀 모델의 RMSE를 측정
  - 사이킷런의 mean_squared_error() 함수에 squared 매개변수를 False로 지정하여 RMSE를 계산

```
>>> from sklearn.metrics import mean_squared_error
>>> lin_rmse = mean_squared_error(housing_labels, housing_predictions,
...                               squared=False)
...
>>> lin_rmse
68687.89176589991
```

2.6 모델 선택과 훈련(3)

- DecisionTreeRegressor를 훈련

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

- 훈련 세트로 평가

```
>>> housing_predictions = tree_reg.predict(housing)
>>> tree_rmse = mean_squared_error(housing_labels, housing_predictions,
...                                squared=False)
...
>>> tree_rmse
0.0
```

2.6 모델 선택과 훈련(4)

2.6.2 교차 검증으로 평가하기

- k-폴드 교차 검증(k-fold cross-validation)
  - 훈련 세트를 폴드 fold라 불리는 중복되지 않은 10개의 서브셋으로 랜덤으로 분할
  - 결정 트리 모델을 10번 훈련하고 평가하는데, 매번 다른 폴드를 선택해 평가에 사용
  - 나머지 9개 폴드는 훈련에 사용됩니다. 10개의 평가 점수가 담긴 배열이 결과

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
```

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     66868.027288
std       2060.966425
min      63649.536493
25%      65338.078316
50%      66801.953094
75%      68229.934454
max      70094.778246
dtype: float64
```

- 결정 트리는 RMSE가 약 66.868이고 표준 편차가 약 2,061
- 선형 회귀 모델에서 이 값을 계산하면 평균 RMSE가 69,858이고 표준 편차가 4,182
- 따라서 결정 트리 모델이 선형 회귀 모델보다 아주 조금 나은 것 같지만 심각한 과대적합 때문에 차이가 미미함
- 훈련 오차가 작고(실제로 0) 검증 오차는 높기 때문에 과대적합

2.6 모델 선택과 훈련(5)

- RandomForestRegressor 모델
  - 랜덤 포레스트는 특성을 랜덤으로 선택해서 많은 결정 트리를 만들고 예측의 평균을 구하는 방식으로 작동
  - 앙상블(ensemble) - 서로 다른 모델들로 구성된 모델
    - 앙상블은 기반 모델(여기서는 결정 트리)의 성능을 높임

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
```

```
>>> pd.Series(forest_rmses).describe()
count       10.000000
mean     47019.561281
std       1033.957120
min      45458.112527
25%      46464.031184
50%      46967.596354
75%      47325.694987
max      49243.765795
dtype: float64
```

- RandomForestRegressor를 훈련하고 훈련 세트에서 RMSE를 측정하면 약 17.474를 얻음 - 매우 낮은 값으로 여전히 많이 과대적합
- 해결 방법
  - 모델 단순화
  - 제한(규제)
  - 더 많은 훈련 데이터 확보

2.7 모델 미세 튜닝(1)

2.7.1 그리드 서치

- GridSearchCV
  - 탐색하고자 하는 하이퍼파라미터와 시도해볼 값을 지정
  - 교차 검증을 사용해 가능한 모든 하이퍼파라미터 조합을 평가

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

2.7 모델 미세 튜닝(2)

- param_grid에는 두 개의 딕셔너리가 있음
  - GridSearchCV가 먼저 첫 번째 딕셔너리에 지정된 n_clusters와 max_features 하이퍼파라미터 값의 3 × 3 = 9개 조합을 모두 평가
  - 그다음 두 번째 딕셔너리에 있는 하이퍼파라미터 값의 2 × 3 = 6개 조합을 모두 평가
  - 따라서 그리드 서치는 총 9 + 6 = 15개의 하이퍼파라미터 조합을 탐색
  - 3-폴드 교차 검증을 사용하므로 각 조합마다 3번씩 파이프라인을 훈련하며, 이는 총 15 × 3 = 45번 훈련이 일어난다는 의미

```
>>> grid_search.best_params_
{'preprocessing__geo__n_clusters': 15, 'random_forest__max_features': 6}
```

2.7 모델 미세 튜닝(3)

- grid_search.cv_results_
  - 속성은 딕셔너리이지만 데이터프레임으로 감싸면 각 하이퍼파라미터 조합과 교차 검증 분할에 대한 테스트 점수와 모든 분할에 대한 평균 테스트 점수를 확인

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_)
>>> cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
>>> [...]        # 페이지에 맞도록 열 이름을 바꾸고 점수를 음수로 바꿉니다.
>>> cv_res.head() # 첫 번째 열은 행 ID입니다.
   n_clusters max_features split0 split1 split2 mean_test_rmse
12         15            6  43460  43919  44748          44042
13         15            8  44132  44075  45010          44406
14         15           10  44374  44286  45316          44659
7          10            6  44683  44655  45657          44999
9          10            6  44683  44655  45657          44999
```

  - 최상의 모델에 대한 평균 테스트 RMSE 점수는 44,042
  - 기본 하이퍼파라미터 값을 사용해 얻은 점수(47,019)보다 좋음

2.7 모델 미세 튜닝(4)

2.7.2 랜덤 서치

- 하이퍼파라미터 탐색 공간이 커지면 RandomizedSearchCV가 종종 선호됨
  - 가능한 모든 조합을 시도하는 대신 각 반복마다 하이퍼파라미터에 임의의 수를 대입하여 지정한 횟수만큼 평가
  - 주요 장점

    1) 하이퍼파라미터 값이 연속적이면(또는 이산적이지만 가능한 값이 많다면) 랜덤 서치를 1,000번 실행했을 때 각 하이퍼파라미터마다 1,000개의 다른 값을 탐색
       - 반면 그리드 서치는 하이퍼파라미터에 대해 나열한 몇 개의 값만을 탐색

    2) 어떤 하이퍼파라미터가 성능 면에서 큰 차이를 만들지 못하지만 아직 이 사실을 모른다고 가정
       - 10 개의 가능한 값이 있을 때 이를 그리드 서치에 추가하면 훈련이 10배 더 오래 걸림
       - 하이퍼파라미터를 랜덤 서치에 추가하면 탐색 시간이 더 늘어나지 않음

    3) 6개의 하이퍼파라미터에 대해 각각 10개의 값을 탐색한다면 그리드 서치는 백만 번 모델을 훈련해야 함
       - 반면 랜덤 서치는 지정한 반복 횟수만큼 실행할 수 있음

2.7 모델 미세 튜닝(5)

- 하이퍼파라미터마다 가능한 값의 리스트나 확률 분포를 제공

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distribs, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

2.7 모델 미세 튜닝(6)

- HalvingRandomSearchCV와 HalvingGridSearchCV 하이퍼파라미터 탐색 클래스
  - 빠르게 훈련하고 대규모 하이퍼파라미터 공간을 탐색하기 위해 계산 자원을 더 효율적으로 사용
  - 작동 방식
    - 첫 번째 반복에서 많은 하이퍼파라미터 조합('후보')이 그리드 서치나 랜덤 서치를 사용해 생성
    - 이 후보들을 사용해 모델을 훈련하고 이전과 같은 방식으로 교차 검증을 사용해 평가
      - 첫 번째 반복의 속도를 높이기 위해 제한된 자원으로 훈련
      - '제한된 자원'이란 모델이 훈련 세트의 작은 일부분에서 훈련된다는 것을 의미
    - 다른 제한도 가능
      - 예) 모델이 훈련 반복 횟수에 관한 하이퍼파라미터를 가지고 있다면 반복 횟수를 줄일 수 있음
      - 모든 후보를 평가한 후에 최상의 후보만 다음 단계로 넘어가 더 많은 자원을 사용
      - 몇 번의 반복이 진행된 후 최종 후보들이 전체 자원을 사용해 평가
      - 하이퍼파라미터 튜닝 시간을 단축

2.7 모델 미세 튜닝(7)

2.7.3 앙상블 방법

- 모델을 세밀하게 튜닝하는 또 다른 방법은 최상의 모델을 연결해보는 것
  - 결정 트리의 앙상블인 랜덤 포레스트가 결정 트리 하나보다 더 성능이 좋은 것처럼 모델의 그룹(또는 앙상블)이 최상의 단일 모델보다 더 나은 성능을 발휘할 때가 많음
    - 특히 개별 모델이 각기 다른 형태의 오차를 만들 때 그러함
    - 예) k-최근접 이웃 모델을 훈련하고 미세 튜닝한 다음 이 모델의 예측과 랜덤 포레스트의 예측을 평균하여 예측으로 삼는 앙상블 모델을 만들 수 있음

2.7 모델 미세 튜닝(8)

2.7.4 최상의 모델과 오차 분석

- RandomForestRegressor는 정확한 예측을 만들기 위한 각 특성의 상대적인 중요도를 제공

```
>>> final_model = rnd_search.best_estimator_ # 전처리 포함됨
>>> feature_importances = final_model["random_forest"].feature_importances_
>>> feature_importances.round(2)
array([0.07, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, [...], 0.01])
```

- 중요도 점수를 내림차순으로 정렬하고 특성 이름과 함께 표시

```
>>> sorted(zip(feature_importances,
...            final_model["preprocessing"].get_feature_names_out()),
...            reverse=True)
...
[(0.18694559869103852, 'log__median_income'),
 (0.0748194905715524, 'cat__ocean_proximity_INLAND'),
 (0.06926417748515576, 'bedrooms__ratio'),
 (0.05446998753775219, 'rooms_per_house__ratio'),
 (0.05262301809680712, 'people_per_house__ratio'),
 (0.03819415873915732, 'geo__Cluster 0 similarity'),
 [...]
 (0.00015061247730531558, 'cat__ocean_proximity_NEAR BAY'),
 (7.301686597099842e-05, 'cat__ocean_proximity_ISLAND')]
```

- 이 정보를 바탕으로 덜 중요한 특성들을 제외
- 예) ocean_proximity 카테고리 중 하나만 실제로 유용하므로 다른 카테고리는 제외

2.7 모델 미세 튜닝(9)

2.7.5 테스트 세트로 시스템 평가하기

- 테스트 세트에서 최종 모델을 평가
  - 테스트 세트의 특성과 레이블을 사용해 final_model을 실행하여 데이터를 변환하고 예측 및 평가

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

final_rmse = mean_squared_error(y_test, final_predictions, squared=False)
print(final_rmse) # 출력: 41424.40026462184
```

2.7 모델 미세 튜닝(10)

- 일반화 오차의 점 추정(point estimate)
  - scipy.stats.t.interval()을 사용해 일반화 오차의 95% 신뢰 구간 confidence interval을 계산
  - 구간은 39,275와 43,467 사이로 꽤 크며 이전 점 추정값인 41,424는 대략 중간에 해당

```
>>> from scipy import stats
>>> confidence = 0.95
>>> squared_errors = (final_predictions - y_test) ** 2
>>> np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
...                          loc=squared_errors.mean(),
...                          scale=stats.sem(squared_errors)))
...
array([39275.40861216, 43467.27680583])
```

2.8 론칭, 모니터링, 시스템 유지 보수(1)

- joblib 라이브러리를 사용해 모델을 저장

```
import joblib

joblib.dump(final_model, "my_california_housing_model.pkl")
```

- 모델이 사용하는 모든 사용자 정의 클래스와 함수를 먼저 임포트(이 코드를 제품 환경으로 전달)
- 모델을 로드하고 예측

```
import joblib
[...] # import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, ...

def column_ratio(X): [...]
def ratio_name(function_transformer, feature_names_in): [...]
class ClusterSimilarity(BaseEstimator, TransformerMixin): [...]

final_model_reloaded = joblib.load("my_california_housing_model.pkl")

new_data = [...] # 예측을 만들려는 새로운 구역
predictions = final_model_reloaded.predict(new_data)
```

2.8 론칭, 모니터링, 시스템 유지 보수(2)

- 모델을 웹 사이트 안에서 사용
  - 사용자가 새로운 구역에 관한 정보 를 입력하고 [가격 예측하기] 버튼을 클릭
  - 이 데이터를 포함한 쿼리 query가 웹 서버로 전송되어 웹 애플리케이션으로 전달
  - 결국 애플리케이션의 코드가 모델의 predict() 메서드를 호출
    (모델을 사용할 때마다 로드하지 않고 서버가 시작될 때 모델을 로드하는 것이 좋음)
  - 또는 웹 애플리케이션이 REST API45를 통해 질의할 수 있는 전용 웹 서비스로 모델을 감쌀 수 있음
    - 주 애플리케이션을 건드리지 않고 모델을 새 버전으로 업그레이드하기 쉬움
  - 필요한 만큼 웹 서비스를 시작하고 웹 애플리케이션에서 웹 서비스로 오는 요청을 로드 밸런싱(load balancing)할 수 있기 때문에 규모를 확장하기도 쉬움
  - 웹 애플리케이션을 파이썬이 아닌 다른 언어로도 작성할 수 있음

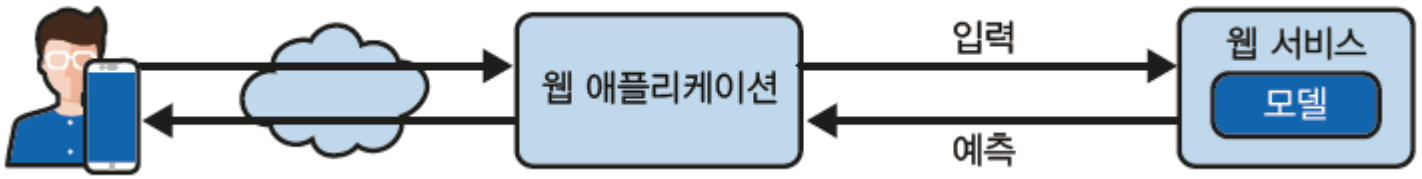

그림 2-20 웹 서비스로 배포된 모델을 웹 애플리케이션에서 사용

2.8 론칭, 모니터링, 시스템 유지 보수(3)

- 모델을 구글 버텍스 Vertex AI(이전 이름은 구글 클라우드 AI 플랫폼과 구글 클라우드 ML 엔진(Google Cloud ML Engine)와 같은 클라우드에 배포
- 모델의 실전 성능 모니터링
- 모델 성능을 평가하기 위해 사람의 분석을 추가
- 데이터가 계속 변화하면 데이터셋을 업데이트하고 모델을 정기적으로 다시 훈련
- 전체 공정 자동화
- 모델의 입력 데이터 품질 평가
- 모든 모델 백업(backup)

2.9 직접 해보세요!

- 머신러닝 프로젝트의 프로세스 요약
  - 데이터 준비 단계
  - 모니터링 도구 구축
  - 사람의 평가 파이프라인 세팅
  - 주기적인 모델 학습 자동화

- 직업 해보기
  - 노트북을 꺼내 관심 있는 데이터셋을 선택해서 A부터 Z까지 전체 프로세스 만들기
  - 캐글(https://kaggle.com) 등 경연 사이트에서 실험

연습문제

1. 서포트 벡터 머신 회귀(sklearn.svm.SVR)를 kernel="linear"(하이퍼파라미터 C를 바꿔가며)나 kernel="rbf"(하이퍼파라미터 C와 gamma를 바꿔가며) 등의 다양한 하이퍼파라미터 설정으로 시도
2. GridSearchCV를 RandomizedSearchCV로 바꿔보기
3. 가장 중요한 특성을 선택하는 SelectFromModel 변환기를 준비 파이프라인에 추가해보기
4. fit() 메서드 안에서 k-최근접 이웃 회귀(sklearn.neighbors.KNeighborsRegressor)를 훈련하고 transform() 메서드에서 이 모델의 예측을 반환하는 사용자 정의 변환기 만들기
5. GridSearchCV를 사용해 준비 단계의 옵션을 자동으로 탐색
6. StandardScalerClone 클래스를 처음부터 다시 구현하고 inverse_transform() 메서드를 추가

연습문제 정답은 https://github.com/rickiepark/handson-ml3에 있는 주피터 노트북에서 확인